

# Computing Directed Pathwidth in $O(1.89^n)$ Time

Kenta Kitsunai<sup>1</sup> · Yasuaki Kobayashi<sup>2</sup> ·  
Keita Komuro<sup>1</sup> · Hisao Tamaki<sup>1</sup> · Toshihiro Tano<sup>1</sup>

Received: 8 October 2014 / Accepted: 2 June 2015 / Published online: 10 June 2015  
© Springer Science+Business Media New York 2015

**Abstract** We give an algorithm for computing the directed pathwidth of a digraph with  $n$  vertices in  $O(1.89^n)$  time. This is the first algorithm with running time better than the straightforward  $O^*(2^n)$ . As a special case, it computes the pathwidth of an undirected graph in the same amount of time, improving on the algorithm due to Suchan and Villanger which runs in  $O(1.9657^n)$  time.

**Keywords** Exact exponential algorithm · Graph algorithm · Pathwidth

## 1 Introduction

The *pathwidth* [2,21] of an undirected graph  $G$  is defined as follows. A *path-decomposition* of  $G$  is a sequence  $\{X_i\}$ ,  $1 \leq i \leq t$ , of vertex sets of  $G$  that satisfies the following three conditions:

1.  $\bigcup_{1 \leq i \leq t} X_i = V(G)$ ,
2. for each edge  $\{u, v\}$  of  $G$ , there is some  $i$ ,  $1 \leq i \leq t$  such that  $u, v \in X_i$ , and
3. for each  $v \in V(G)$ , the set of indices  $i$  such that  $v \in X_i$  is contiguous, i.e., is of the form  $\{i \mid a \leq i \leq b\}$ .

The *width* of a path-decomposition  $\{X_i\}$ ,  $1 \leq i \leq t$ , is  $\max_{1 \leq i \leq t} |X_i| - 1$  and the *pathwidth* of  $G$  is the smallest integer  $k$  such that there is a path-decomposition of  $G$  whose width is  $k$ .

---

✉ Yasuaki Kobayashi  
yasuaki.kobayashi@gakushuin.ac.jp

<sup>1</sup> Department of Computer Science, Meiji University, Kawasaki-shi, Kanagawa, Japan

<sup>2</sup> Computer Center, Gakushuin University, Toshima-ku, Tokyo, Japan

The *directed path-decomposition* of a digraph  $G$  is defined analogously. A sequence  $\{X_i\}$ ,  $1 \leq i \leq t$ , of vertex sets is a directed path-decomposition of  $G$  if, together with conditions 1 and 3 above, the following condition 2' instead of condition 2 is satisfied:

- 2'. for each directed edge  $(u, v)$  of  $G$ , there is a pair  $i, j$  of indices such that  $i \leq j$ ,  $u \in X_i$ , and  $v \in X_j$ .

The directed pathwidth of  $G$  is defined similarly to the pathwidth of an undirected graph. According to Barát [1], the notion of directed pathwidth was introduced by Reed, Thomas, and Seymour around 1995.

For an undirected graph  $G$ , let  $\hat{G}$  denote the digraph obtained from  $G$  by replacing each edge  $\{u, v\}$  by a pair of directed edges  $(u, v)$  and  $(v, u)$ . Then, condition 2 for  $G$  implies condition 2' for  $\hat{G}$  and, conversely, condition 2' for  $\hat{G}$  together with condition 3 implies condition 2 for  $G$ . Therefore, a directed path-decomposition of  $\hat{G}$  is a path-decomposition of  $G$  and vice versa. Thus, the problem of computing the pathwidth of an undirected graph is a special case of the problem of computing the directed pathwidth of a digraph. On the other hand, directed pathwidth is of interest since some problems, such as Directed Hamiltonicity, are polynomial time solvable on digraphs with bounded directed pathwidth [13] although not necessarily on those whose underlying graphs have bounded pathwidth. Directed pathwidth is also studied in the context of search games [1, 26].

Computing pathwidth is NP-hard [14] even for bounded degree planar graphs [19], chordal graphs [11], cocomparability graphs [12] and bipartite distance-hereditary graphs [18] (although it is polynomial time solvable for permutation graphs [5], cographs [6], and circular-arc graphs [23]). Consequently, computing directed pathwidth is NP-hard even for digraphs whose underlying graphs lie in these classes. On the positive side, pathwidth is fixed parameter tractable [22] with running time linear in the number of vertices [3]. In contrast, it is open whether directed pathwidth is fixed parameter tractable. However, directed pathwidth does admit XP algorithms [20, 25], that is, algorithms with running time  $n^{O(k)}$ , where  $n$  is the number of vertices and  $k$  is the directed pathwidth. The algorithm in [25], due to one of the present authors, is claimed to run in  $O(mn^{k+1})$  time, where  $m$  is the number of edges. Unfortunately, this algorithm as presented in [25] is flawed. Since this algorithm is an essential ingredient of our current result, we develop a corrected version in this paper, which runs in  $O(\frac{mn^{2k}}{(k-1)!})$  time. The algorithm due to Nagamochi [20] solves more general linear layout problems for submodular systems and, when specialized to directed pathwidth, runs in  $O(kmn^{2k})$  time. Although the running time of his algorithm is similar to ours, it does not seem that his algorithm can replace the role of our XP algorithm in the current work.

Without parameterization, both directed and undirected pathwidth can be solved in  $O^*(2^n)$  time, where  $n$  is the number of vertices and the  $O^*$  notation hides polynomial factors, using Bellman–Held–Karp style dynamic programming for vertex ordering problems (see [4]). Similarly, some undirected graph parameters related to pathwidth, such as treewidth, cutwidth, and treedepth can be computed in  $O^*(2^n)$  time. As for treewidth and treedepth, there are algorithms that run in  $O^*(1.73461^n)$  time [10] and in  $O^*(1.9602^n)$  time [8], respectively, although no algorithm is known for cutwidth whose running time is  $O^*(c^n)$  for some  $c < 2$ .

For the computation of pathwidth, Suchan and Villanger [24] improved the running time to  $O(1.9657^n)$  and also gave an additive constant approximation of pathwidth in  $O(1.89^n)$  time. On the other hand, no algorithm faster than  $O^*(2^n)$  time was known for directed pathwidth before the present work.

Our result is as follows.

**Theorem 1** *The directed pathwidth of a digraph with  $n$  vertices can be computed in  $O(1.89^n)$  time.*

Our algorithm can be viewed as one based on Bellman-Held-Karp style dynamic programming. For each  $U \subseteq V(G)$ , let  $N^-(U)$  denote the set of in-neighbors of  $U$ . Given a positive integer  $k$  and a digraph  $G$ , we build the collection of “feasible” subsets of  $V(G)$ , where  $U \subseteq V(G)$  is considered feasible if  $G[U \cup N^-(U)]$  has a directed path-decomposition of width at most  $k$  whose last subset  $X_t$  contains  $N^-(U)$ . To get a non-trivial bound on the number of subsets  $U$  to be considered, we adopt a strategy essentially due to Suchan and Villanger [24]. Since either  $U$ ,  $N^-(U)$ , or  $V(G) \setminus (U \cup N^-(U))$  has cardinality at most  $n/3$ , the hope is that we may be able to obtain a non-trivial upper bound on the number of relevant subsets  $U$  using the well-known upper bound  $2^{H(\alpha)n}$  on the number of subsets of  $V(G)$  with cardinality at most  $\alpha n$ , where  $H(x)$  is the binary entropy function. Setting  $\alpha = \frac{1}{3}$ , this upper bound is  $O(1.89^n)$  since  $2^{H(1/3)} < 1.89$ . The difficulty, as observed in [24], is that there may be an exponential number of subsets  $U$  with  $N^-(U) = S$  for each  $S$ . The larger constant 1.9567 or the relaxation to additive constant approximation in [24] comes from the need to deal with this problem.

A key to overcoming this difficulty is the following observation whose undirected version is used in [24]. Suppose we are to decide whether the directed pathwidth of  $G$  is at most  $k$  and  $S$  is a vertex set of  $G$  with  $|S| \leq k - g$ , where  $g$  is some positive integer. Let  $\mathcal{C}$  be the set of strongly connected components of  $G[V(G) \setminus S]$  with cardinality greater than  $g$ . For each subset  $\mathcal{A}$  of  $\mathcal{C}$ , we are able to define at most one subset  $U$  in a “canonical form” such that  $N^-(U) = S$ ,  $U$  contains all components in  $\mathcal{A}$ , but disjoint from all components in  $\mathcal{C} \setminus \mathcal{A}$ . Although there can be variants of  $U$  that satisfy these conditions, it can be shown that only the canonical one is needed in our dynamic programming computation. Thus, the number of relevant subsets  $U$  with  $N^-(U) = S$  is bounded by  $2^{|\mathcal{C}|} \leq 2^{n/g}$ . See Lemma 4 for a more formal treatment.

Our result is established by two algorithms, which we call LARGE-WIDTH and SMALL-WIDTH. Algorithm LARGE-WIDTH deals with the case where  $k \geq (\frac{1}{3} + \delta)n$ , where  $\delta$  is a small constant, while algorithm SMALL-WIDTH deals with the other case. In algorithm LARGE-WIDTH, the slack of  $\delta n$  allows us to bound the number of subsets  $U$  with  $N^-(U) = S$  for each  $S$  with  $|S| \leq n/3$  by  $2^{1/\delta}$ . In algorithm SMALL-WIDTH, we force a slack of  $g$  where  $g$  is a large enough constant: we record only those feasible sets  $U$  with  $|N^-(U)| \leq k - g$ . To process those subsets  $U$  with  $|N^-(U)| > k - g$ , we extend the corrected XP algorithm to an algorithm which runs in  $n^{2g+O(1)}$  time and either decides that the directed pathwidth of  $G$  is at most  $k$ , decides that  $U$  is irrelevant, or produces some proper superset  $W$  of  $U$  with  $|N^-(W)| \leq k - g$ , which can safely replace  $U$  in the search. See Lemma 7 for details. Choosing  $\delta$  and  $g$

so that  $2^{H(1/3+\delta)+\frac{1}{g}} < 1.89$ , the number of relevant subsets multiplied by the time for processing each of them is bounded by  $O(1.89^n)$ .

The key notion of our algorithm is *commitment* described in Sect. 3, which is also used in experimental results [7, 17].

The preliminary version of this paper appeared in [16], which relied on the flawed XP algorithm in [25]. In the present version, as mentioned above, this reliance is replaced by a self-contained exposition of the corrected XP algorithm.

The rest of this paper is organized as follows. In Sect. 2, we define basic concepts and prove some lemmas needed by our algorithms. In Sect. 3, we describe the notion of commitments, our general principle for pruning the search space, and prove related lemmas. In Sect. 4, we state and analyze algorithm LARGE-WIDTH. In Sect. 5, we state and analyze algorithm SMALL-WIDTH, assuming the XP algorithm. In Sect. 6, we combine the two algorithms to prove Theorem 1. Finally, in Sect. 7, we describe the corrected XP algorithm and its extension assumed in Sect. 5.

## 2 Preliminaries

Let  $G$  be a digraph. We use the standard notation:  $V(G)$  is the set of vertices of  $G$ ,  $E(G)$  is the set of edges of  $G$ , and  $G[U]$ , where  $U \subseteq V(G)$ , is the subgraph of  $G$  induced by  $U$ . For each vertex  $v \in V(G)$ , we denote by  $N_G^-(v)$  the set of in-neighbors of  $v$ , i.e.,  $N_G^-(v) = \{u \in V(G) \setminus \{v\} \mid (u, v) \in E(G)\}$ . For each subset  $U$  of  $V(G)$ , we denote by  $N_G^-(U)$  the set of in-neighbors of  $U$ , i.e.,  $N_G^-(U) = \bigcup_{v \in U} N_G^-(v) \setminus U$ . We denote in-degrees  $|N_G^-(v)|$  and  $|N_G^-(U)|$  by  $d_G^-(v)$  and  $d_G^-(U)$  respectively. When  $G$  is clear from the context, we drop  $G$  from this notation. We also use the notation  $\tilde{U} = V(G) \setminus (U \cup N^-(U))$  where  $G$  is implicit.

The following fact is well-known and is straightforward to prove.

**Proposition 1** *The integer valued function  $d^-$  on  $2^{V(G)}$  is submodular. That is, for every pair  $X, Y \subseteq V(G)$ , we have  $d^-(X) + d^-(Y) \geq d^-(X \cup Y) + d^-(X \cap Y)$ .*

When we consider a sequence  $\sigma$  of vertices of  $G$  in this paper,  $\sigma$  will always be such that each vertex of the graph occurs at most once in  $\sigma$ . For each sequence  $\sigma$  of vertices of  $G$ , we denote by  $V(\sigma)$  the set of vertices constituting  $\sigma$  and by  $|\sigma| = |V(\sigma)|$  the length of  $\sigma$ . A sequence  $\sigma$  of vertices of  $G$  is a *permutation* if  $V(\sigma) = V(G)$ . For brevity, we often write  $d^-(\sigma)$  and  $N^-(\sigma)$  for  $d^-(V(\sigma))$  and  $N^-(V(\sigma))$ , respectively.

For each pair of sequences  $\sigma$  and  $\tau$  of vertices of  $G$  such that  $V(\sigma) \cap V(\tau) = \emptyset$ , we denote by  $\sigma\tau$  the sequence that is  $\sigma$  followed by  $\tau$ . If  $\sigma' = \sigma\tau$  for some  $\tau$ , then we say that  $\sigma$  is a *prefix* of  $\sigma'$  and that  $\sigma'$  is an *extension* of (or *extends*)  $\sigma$ ; we say that  $\sigma$  is a *proper prefix* of  $\sigma'$  and that  $\sigma'$  is a *proper extension* of  $\sigma$  if  $\tau$  is nonempty. We call  $\sigma$  a *subsequence* of  $\tau$ , if  $V(\sigma) \subseteq V(\tau)$  and moreover the order in which the vertices of  $\sigma$  appear in  $\tau$  is the same as in  $\sigma$ .

Let  $G$  be a digraph and  $k$  a positive integer. We say a sequence  $\sigma$  of vertices of  $G$  is *k-feasible* for  $G$  if  $d^-(\sigma') \leq k$  for every prefix  $\sigma'$  of  $\sigma$ . We say that  $\sigma$  is *strongly k-feasible* for  $G$  if  $\sigma$  is a prefix of a *k-feasible* permutation. Clearly, if  $\sigma$  is strongly *k-feasible* then it is *k-feasible*. We may drop the reference to  $G$  and say  $\sigma$  is *k-feasible* (or *strongly k-feasible*) when  $G$  is clear from the context. These feasibility notions

are defined for vertex sets as well. For each  $U \subseteq V(G)$ , we say that  $U$  is  $k$ -feasible (strongly  $k$ -feasible) if there is a  $k$ -feasible (strongly  $k$ -feasible) sequence  $\sigma$  with  $V(\sigma) = U$ .

The *directed vertex separation number* of digraph  $G$ , denoted by  $\text{dvsn}(G)$ , is the minimum integer  $k$  such that  $V(G)$  is  $k$ -feasible.

It is known that the directed pathwidth of  $G$  equals  $\text{dvsn}(G)$  for every digraph  $G$  [26] (see also [15] for the undirected case). Based on this fact, we work on the directed vertex separation number in the remaining of this paper.

Let  $H(x) = -x \log x - (1-x) \log(1-x)$ ,  $0 < x < 1$ , denote the binary entropy function. We freely use the following well-known bound on the number of subsets of bounded cardinality.

**Proposition 2** (see [9], for example) *Let  $S$  be a set of  $n$  elements and let  $0 < \alpha \leq \frac{1}{2}$ . Then the number of subsets of  $S$  with cardinality at most  $\alpha n$  is at most  $2^{H(\alpha)n}$ .*

### 3 Commitments and Related Lemmas

We use several techniques to reduce the search space for a  $k$ -feasible permutation. They can be treated in a unified manner based on a general principle called *commitments* in [25].

Let  $\sigma$  be a  $k$ -feasible sequence and let  $\tau$  be a  $k$ -feasible proper extension of  $\sigma$ . We call  $\tau$  a  $k$ -committable extension of  $\sigma$  if, for every  $X$  with  $V(\sigma) \subseteq X \subseteq V(\tau)$ , we have  $d^-(X) \geq d^-(\tau)$ . We extend this notion to vertex sets:  $W \subseteq V(G)$  is a  $k$ -committable extension of  $U \subseteq V(G)$  if there are sequences  $\sigma$  and  $\tau$  with  $U = V(\sigma)$  and  $W = V(\tau)$  such that  $\tau$  is a  $k$ -committable extension of  $\sigma$ .

The following lemma states that, if a  $k$ -feasible sequence  $\sigma$  has a  $k$ -committable extension  $\tau$ , then, in search for a  $k$ -feasible permutation that extends  $\sigma$ , the only extensions of  $\sigma$  to be considered are the extensions of  $\tau$ . In other words, we may let the search node  $\sigma$  commit to  $\tau$ .

**Lemma 1** (Commitment Lemma [25]) *Let  $G$  be a digraph and  $k$  a positive integer. Let  $\sigma$  be a  $k$ -feasible sequence and let  $\tau$  be a  $k$ -committable extension of  $\sigma$ . If  $\sigma$  is strongly  $k$ -feasible then so is  $\tau$ .*

*Proof* Let  $\sigma$  and  $\tau$  be as above and suppose  $\sigma$  is strongly  $k$ -feasible. Let  $\eta$  be a  $k$ -feasible permutation that extends  $\sigma$ . Let  $\alpha$  be the subsequence of  $\eta$  such that  $V(\alpha) = V(G) \setminus V(\tau)$ . Our goal is to show that the permutation  $\tau\alpha$  is  $k$ -feasible.

For  $1 \leq i \leq |\alpha|$ , let  $\alpha_i$  denote the prefix of  $\alpha$  of length  $i$  and let  $\eta_i$  denote the maximal prefix of  $\eta$  such that  $V(\eta_i) \cap V(\alpha) = V(\alpha_i)$ . Fix  $i$ ,  $1 \leq i \leq |\alpha|$ . By the submodularity of  $d^-$ , we have

$$d^-(\tau) + d^-(\eta_i) \geq d^-(V(\tau) \cap V(\eta_i)) + d^-(V(\tau) \cup V(\eta_i)).$$

Since  $V(\sigma)$  is disjoint from  $V(\alpha)$ ,  $\eta_i$  is an extension of  $\sigma$  and hence  $V(\sigma) \subseteq V(\tau) \cap V(\eta_i) \subseteq V(\tau)$ . Therefore, by the definition of  $k$ -committable extensions, we have  $d^-(V(\tau) \cap V(\eta_i)) \geq d^-(\tau)$  and hence

$$\begin{aligned} d^-(\tau\alpha_i) &= d^-(V(\tau) \cup V(\eta_i)) \\ &\leq d^-(\eta_i). \end{aligned}$$

Since  $d^-(\eta_i) \leq k$  as  $\eta$  is  $k$ -feasible, we have  $d^-(\tau\alpha_i) \leq k$ . Since this holds for every  $i$ ,  $1 \leq i \leq |\alpha|$ ,  $\tau\alpha$  is  $k$ -feasible.  $\square$

**Corollary 1** *Let  $G$  be a digraph and  $k$  a positive integer. Let  $U$  be a  $k$ -feasible subset of  $V(G)$  and let  $W$  be a  $k$ -committable extension of  $U$ . If  $U$  is strongly  $k$ -feasible then so is  $W$ .*

The XP algorithm in Sect. 7 uses  $k$ -committable extensions in general form, while our main algorithms in Sect. 4 and Sect. 5 directly use a few special cases of  $k$ -committable extensions introduced below.

Following the terminology of [24] for the undirected case, we call  $U \subseteq V(G)$  a *full set* (with respect to  $G$ ), if there is no  $v \in N^-(U)$  with  $N^-(v) \subseteq U \cup N^-(U)$ . For each  $U$ , there is a unique superset of  $U$  that is a full set, which we denote by  $\text{fullset}(U)$ . Indeed,  $\text{fullset}(U)$  is defined to be  $U \cup Y$  where

$$Y = \{v \in N^-(U) \mid N^-(v) \subseteq U \cup N^-(U)\}.$$

**Proposition 3** *Let  $U \subseteq V(G)$  be  $k$ -feasible. Then, either  $\text{fullset}(U) = U$  or  $\text{fullset}(U)$  is a  $k$ -committable extension of  $U$ .*

*Proof* Suppose  $\text{fullset}(U)$  is a proper superset of  $U$ . Let  $\sigma$  be an arbitrary  $k$ -feasible sequence with  $V(\sigma) = U$  and let  $\tau$  be an arbitrary extension of  $\sigma$  with  $V(\tau) = \text{fullset}(U)$ . For each extension  $\eta$  of  $\sigma$  that is a prefix of  $\tau$ , we have  $N^-(\eta) \subseteq N^-(U)$ . It follows that  $\tau$  is  $k$ -feasible. For each  $X$  such that  $U \subseteq X \subseteq \text{fullset}(U)$ , we have  $N^-(X) = N^-(U) \setminus X \supseteq N^-(U) \setminus \text{fullset}(U) = N^-(\text{fullset}(U))$  and therefore  $d^-(X) \geq d^-(\text{fullset}(U))$ . This verifies that, by Lemma 1,  $\tau$  is a  $k$ -committable extension of  $U$ .  $\square$

Let  $U \subseteq V(G)$  and let  $H = G[V(G) \setminus N^-(U)]$  be the subgraph of  $G$  induced by the complement of  $N^-(U)$ . Observe that, for each strongly connected component  $C$  of  $H$ , either  $C \subseteq U$  or  $C \subseteq \tilde{U}$ , as  $H$  does not contain an edge from  $\tilde{U}$  to  $U$ . Let  $\mathcal{C}(H)$  denote the set of all strongly connected components of  $H$  and consider the natural partial ordering  $<$  on  $\mathcal{C}(H)$ :  $C < D$  if and only if  $C \neq D$  and  $H$  contains a directed path from a vertex in  $C$  to a vertex in  $D$ .

**Lemma 2** *Let  $U$  and  $H$  be as above and let  $C$  be a strongly connected component of  $H$  contained in  $\tilde{U}$  minimal with respect to  $<$ , that is,  $C \in \mathcal{C}(H)$ ,  $C \subseteq \tilde{U}$ , and there is no  $D \in \mathcal{C}(H)$  with  $D \subseteq \tilde{U}$  and  $D < C$ . Suppose furthermore that  $U$  is  $k$ -feasible and  $d^-(U) + |C| \leq k + 1$ . Then,  $U \cup C$  is a  $k$ -committable extension of  $U$ . Moreover, given a  $k$ -feasible sequence  $\sigma$  with  $V(\sigma) = U$ , we can find a  $k$ -committable extension  $\tau$  of  $\sigma$  with  $V(\tau) = U \cup C$  in polynomial time.*

*Proof* Let  $\sigma$  be an arbitrary  $k$ -feasible sequence with  $V(\sigma) = U$  and let  $\tau$  be an arbitrary extension of  $\sigma$  with  $V(\tau) = U \cup C$ . Observe that the minimality condition

for  $C$  implies that  $N^-(U \cup C) = N^-(U)$ . For each proper extension  $\eta$  of  $\sigma$  that is a prefix of  $\tau$ , we have  $N^-(\eta) \subseteq N^-(U) \cup (C \setminus V(\eta))$  and therefore  $d^-(\eta) \leq d^-(U) + |C| - 1 \leq k$ . Therefore,  $\tau$  is  $k$ -feasible. Moreover, for each  $Y \subseteq C$ , we have  $d^-(U \cup Y) \geq d^-(U \cup C)$  since  $N^-(U \cup Y)$  contains  $N^-(U) = N^-(U \cup C)$ . Therefore, by Lemma 1,  $\tau$  is a  $k$ -committable extension of  $\sigma$ .  $\square$

This lemma together with the commitment lemma implies that, under the assumptions of the lemma,  $U \cup C$  is strongly  $k$ -feasible if  $U$  is. The undirected counterpart of this observation is called the *component push rule* in [24].

Based on the partial ordering  $<$  on  $\mathcal{C}(H)$  defined above, we call a subset  $\mathcal{D}$  of  $\mathcal{C}(H)$  *downward closed* if  $C \in \mathcal{D}$  implies  $D \in \mathcal{D}$  for every  $D < C$ .

**Proposition 4** *Let  $U \subseteq V(G)$  and let  $H = G[V(G) \setminus N^-(U)]$ . Then, the set of all strongly connected components of  $H$  contained in  $U$  is a downward closed subset of  $\mathcal{C}(H)$ .*

For each  $U \subseteq V(G)$  with  $d^-(U) \leq k$ , we define  $\text{push}_k(U) = \bigcup_{C \in \mathcal{D}} C$ , where  $\mathcal{D}$  is the maximal downward closed subset of  $\mathcal{C}(H)$  that avoids every  $C \in \mathcal{C}(H)$  with  $C \subseteq \tilde{U}$  and  $d^-(U) + |C| > k + 1$ . Since the set of all strongly connected components of  $H$  contained in  $U$  is downward closed by Proposition 4, we have  $U \subseteq \text{push}_k(U)$ . Algorithmically,  $\text{push}_k(U)$  can be obtained as the value of  $W$  after the following iteration: let  $W = U$  and, as long as the subset of  $\mathcal{C}(H)$  consisting of all the components contained in  $\tilde{W}$  has a minimal element  $C$  with  $d^-(U) + |C| \leq k + 1$ , update  $W$  to be  $W \cup C$ . By a repeated application of Lemma 2, we obtain the following lemma.

**Lemma 3** *Let  $U \subseteq V(G)$ . If  $U$  is  $k$ -feasible then so is  $\text{push}_k(U)$ . Moreover, if  $U$  is strongly  $k$ -feasible then so is  $\text{push}_k(U)$ . Moreover, given a  $k$ -feasible sequence  $\sigma$  with  $V(\sigma) = U$ , we can find a  $k$ -committable extension  $\tau$  of  $\sigma$  with  $V(\tau) = \text{push}_k(U)$  in polynomial time when  $\text{push}_k(U)$  is a proper superset of  $U$ .*

Thus, our algorithm may focus only on those subsets  $U$  with  $\text{push}_k(U) = U$ . The following lemma is used to bound the number of such subsets.

**Lemma 4** *Let  $S \subseteq V(G)$  with  $|S| \leq k$ . Then, the number of vertex sets  $U \subseteq V(G)$  with  $N^-(U) = S$  and  $\text{push}_k(U) = U$  is at most  $2^{\frac{n}{s+1}}$  where  $s = k - |S| + 1$ .*

*Proof* Fix  $S \subseteq V(G)$  with  $|S| \leq k$  and let  $s = k - |S| + 1$ . Let  $H = G[V(G) \setminus S]$  and  $\mathcal{C}_s$  the set of strongly connected components of  $H$  with cardinality strictly larger than  $s$ . As observed before, for each  $U \subseteq V(G)$  with  $N^-(U) = S$  and a strongly connected component  $C$  of  $H$ , either  $C \subseteq U$  or  $C \subseteq \tilde{U}$ . Thus, each such  $U$  induces a bipartition  $(\mathcal{A}_U, \mathcal{B}_U)$  of  $\mathcal{C}_s$ :  $C \subseteq U$  for each  $C \in \mathcal{A}_U$  and  $C \subseteq \tilde{U}$  for each  $C \in \mathcal{B}_U$ .

We call a bipartition  $(\mathcal{A}, \mathcal{B})$  of  $\mathcal{C}_s$  *valid* if there is a downward closed subset  $\mathcal{D}$  of  $\mathcal{C}(H)$  such that  $\mathcal{A} \subseteq \mathcal{D}$  and  $\mathcal{B} \cap \mathcal{D} = \emptyset$ . By Proposition 4,  $(\mathcal{A}, \mathcal{B})$  must be a valid partition in order for some  $U$  to exist such that  $\mathcal{A}_U = \mathcal{A}$  and  $\mathcal{B}_U = \mathcal{B}$ . Furthermore, for each valid bipartition  $(\mathcal{A}, \mathcal{B})$  of  $\mathcal{C}_s$ , there is exactly one  $U \subseteq V(G)$  such that  $N^-(U) = S$ ,  $\text{push}_k(U) = U$ , and  $(\mathcal{A}_U, \mathcal{B}_U) = (\mathcal{A}, \mathcal{B})$ . Indeed, such  $U$  is defined by  $\bigcup_{C \in \mathcal{D}} C$ , where  $\mathcal{D}$  is the maximal downward closed subset of  $\mathcal{C}(H)$  such that  $\mathcal{A} \subseteq \mathcal{D}$  and  $\mathcal{B} \cap \mathcal{D} = \emptyset$ .

Since  $|\mathcal{C}_s| \leq \frac{n}{s+1}$ , the number of bipartitions of  $\mathcal{C}_s$  is at most  $2^{\frac{n}{s+1}}$  and this bound applies to the number of vertex sets  $U$ .  $\square$

## 4 Algorithm LARGE-WIDTH

Given an integer  $k > 0$  and a digraph  $G$  with  $n$  vertices, Algorithm LARGE-WIDTH decides whether  $\text{dvsn}(G) \leq k$  in the following steps.

The algorithm uses function  $f^*$  defined as follows. Define  $f : 2^{V(G)} \rightarrow 2^{V(G)}$  by  $f(U) = \text{fullset}(\text{push}_k(U))$ . Recall that  $\text{fullset}(U) = U \cup \{v \in N^-(U) \mid N^-(v) \subseteq U \cup N^-(U)\}$ . Since  $U \subseteq f(U)$ , there is some  $h$  for each  $U$  such that  $f^h(U) = f^{h+1}(U)$ . We denote this  $f^h(U)$  by  $f^*(U)$ . Note that if  $W = f^*(U)$  for some  $U$  then  $W = \text{fullset}(W) = \text{push}_k(W)$ .

1. Set  $\mathcal{U}_0 := \{\emptyset\}$  and  $\mathcal{U}_i := \emptyset$  for  $1 \leq i \leq n$ .
2. Repeat the following for  $i = 0, 1, \dots, n-1$ .
  - (a) For each  $U \in \mathcal{U}_i$  and for each  $v \in V(G) \setminus U$  with  $d^-(U \cup \{v\}) \leq k$ , let  $W = f^*(U \cup \{v\})$  and update  $\mathcal{U}_j := \mathcal{U}_j \cup \{W\}$  where  $j = |W|$ .
3. If  $\mathcal{U}_n$  is not empty then answer “YES”; otherwise answer “NO”.

**Lemma 5** *Algorithm LARGE-WIDTH is correct.*

*Proof* In the following proof, we abuse the notation and let  $\mathcal{U}_i$ ,  $0 \leq i \leq n$ , denote the final value of the program variable  $\mathcal{U}_i$ . To justify the answer at step 3, we show that  $V(G)$  is  $k$ -feasible if and only if  $\mathcal{U}_n = \{V(G)\}$ .

First note that, by Proposition 3 and Lemma 3, every member of  $\mathcal{U}_i$  is  $k$ -feasible for  $1 \leq i \leq n$ . Therefore if  $\mathcal{U}_n = \{V(G)\}$  then  $V(G)$  is  $k$ -feasible.

To prove the other direction, suppose  $V(G)$  is  $k$ -feasible. We show that  $\mathcal{U}_n = \{V(G)\}$ . First observe that  $\mathcal{U}_0$  contains a member, the empty set, that is strongly  $k$ -feasible. Suppose now that  $\mathcal{U}_i$  for some  $i < n$  contains a strongly  $k$ -feasible member  $U$ . Then, there is some  $v \in V(G) \setminus U$  such that  $U \cup \{v\}$ , and hence  $f^*(U \cup \{v\})$ , is strongly  $k$ -feasible. The algorithm puts this strongly  $k$ -feasible set to  $\mathcal{U}_j$  for some  $j > i$ . By a straightforward induction, we see that  $V(G)$  is put into  $\mathcal{U}_n$ .  $\square$

We analyse the complexity of this algorithm for particular values of  $k$  for which this algorithm is intended.

**Lemma 6** *Let  $\delta > 0$ . For  $k > (\frac{1}{3} + \delta)n$ , algorithm LARGE-WIDTH runs in  $O^*(2^{H(\frac{1}{3})n + \frac{1}{\delta}})$  time.*

*Proof* Let  $\mathcal{W}$  denote the set  $\bigcup_{1 \leq i \leq n} \mathcal{U}_i$  after the algorithm execution. We consider the following subsets of  $\mathcal{W}$ :

$$\begin{aligned} \mathcal{W}_1 &= \{U \in \mathcal{W} \mid |U| \leq n/3\}, \\ \mathcal{W}_2 &= \{U \in \mathcal{W} \mid |U \cup N^-(U)| \geq 2n/3\}, \text{ and} \\ \mathcal{W}_3 &= \mathcal{W} \setminus (\mathcal{W}_1 \cup \mathcal{W}_2). \end{aligned}$$

By Proposition 2, we have  $|\mathcal{W}_1| \leq 2^{H(\frac{1}{3})n}$ . To bound  $|\mathcal{W}_2|$ , observe that  $U = \text{fullset}(U)$  for each member  $U$  of  $\mathcal{W}$  and hence, for each  $S \subseteq V(G)$ , there is at most

one  $U \in \mathcal{W}$  such that  $U \cup N^-(U) = S$ . Since the number of  $S$  with  $|S| \geq 2n/3$  is at most  $2^{H(\frac{1}{3})n}$ , we have  $|\mathcal{W}_2| \leq 2^{H(\frac{1}{3})n}$ .

For each  $U \in \mathcal{W}_3$ , we have  $d^-(U) < n/3$ . For each  $S \subseteq V(G)$  with  $|S| < n/3$ , let  $\mathcal{W}_S = \{U \in \mathcal{W}_3 \mid N^-(U) = S\}$ . Since  $U = \text{push}_k(U)$  for each  $U \in \mathcal{W}$ , we have, by Lemma 4,  $|\mathcal{W}_S| \leq 2^{\frac{n}{k-|S|+2}} \leq 2^{\frac{1}{8}}$  for each  $S$ . Since the number of  $S \subseteq V(G)$  with  $|S| < n/3$  is at most  $2^{H(\frac{1}{3})n}$ , we have  $|\mathcal{W}_3| = O(2^{H(\frac{1}{3})n + \frac{1}{8}})$ . Since each member of  $\mathcal{W}$  is involved in a computation of  $n^{O(1)}$  time, the total running time of the algorithm is  $O^*(2^{H(\frac{1}{3})n + \frac{1}{8}})$ .  $\square$

## 5 Algorithm SMALL-WIDTH

Fix  $\epsilon > 0$  and fix an integer  $g > 1/\epsilon$ . The following description of our algorithm depends on  $g$ . We assume  $k$ , an input to the algorithm, satisfies  $k > g$ ; otherwise the XP algorithm in Sect. 7 runs in  $n^{O(1)}$  time.

Our strategy is to record only those sets  $U$  with  $d^-(U) \leq k - g$  and  $U = \text{push}_k(U)$  in our computation. For each  $S$  with  $|S| \leq k - g$ , by Lemma 4, the number of  $U$  such that  $N^-(U) = S$  and  $U = \text{push}_k(U)$  is at most  $2^{\epsilon n}$ . To process  $U$  with  $d^-(U) > k - g$ , we rely on the following lemma.

**Lemma 7** *There is an algorithm which, given a  $k$ -feasible sequence  $\sigma$  of  $G$  and a non-negative integer  $g$  with  $d^-(\sigma) \geq k - g$ , runs in  $n^{2g + O(1)}$  time and either*

1. *proves that  $\sigma$  is strongly  $k$ -feasible,*
2. *proves that  $\sigma$  is not strongly  $k$ -feasible, or*
3. *produces a  $k$ -committable extension  $\tau$  of  $\sigma$  with  $d^-(\tau) \leq k - g$ .*

The algorithm claimed in this lemma is essentially an XP algorithm for directed pathwidth, along the line of [25]. We defer the proof of this lemma to Sect. 7.

Algorithm SMALL-WIDTH, given  $G$  and  $k$ , decides if  $\text{dvsn}(G) \leq k$  in the following steps. It uses program variables  $\mathcal{S}_i$ ,  $0 \leq i < n$ , each representing a set of sequences of length  $i$ .

1. Set  $\mathcal{S}_0$  to be a singleton set consisting of the empty sequence and set  $\mathcal{S}_i := \emptyset$  for  $1 \leq i < n$ .
2. Repeat the following for  $i = 0, 1, \dots, n - 1$ .  
 For each  $\sigma \in \mathcal{S}_i$  and for each  $v \in V(G) \setminus V(\sigma)$  with  $d^-(\sigma v) \leq k$ , do the following.
  - (a) If  $d^-(\sigma v) \leq k - g$  then let  $W = \text{push}_k(V(\sigma v))$ . If  $W = V(G)$  then answer “YES” and stop. Otherwise, compute a  $k$ -feasible extension  $\tau$  of  $\sigma v$  with  $V(\tau) = W$ , and add  $\tau$  to  $\mathcal{S}_{|\tau|}$  unless there is already some  $\tau' \in \mathcal{S}_{|\tau|}$  with  $V(\tau') = V(\tau)$ .
  - (b) If  $d^-(\sigma v) > k - g$  then apply the algorithm of Lemma 7 to  $\sigma v$ .
    - i. If  $\sigma v$  is found strongly  $k$ -feasible, then stop the entire algorithm answering “YES”.
    - ii. If  $\sigma v$  is found not strongly  $k$ -feasible, then do nothing.
    - iii. Suppose a  $k$ -committable extension  $\tau$  of  $\sigma v$  with  $d^-(\tau) \leq k - g$  is returned. Let  $W = \text{push}_k(V(\tau))$ . If  $W = V(G)$  then answer “YES” and

stop. Otherwise, by Lemma 3, compute a  $k$ -feasible extension  $\tau'$  of  $\tau$  with  $V(\tau') = W$  and add  $\tau'$  to  $\mathcal{S}_{|\tau'|}$  unless there is already some  $\tau'' \in \mathcal{S}_{|\tau'|}$  with  $V(\tau'') = V(\tau')$ .

3. Answer “NO”.

**Lemma 8** *Algorithm SMALL-WIDTH is correct.*

*Proof* As a consequences of Lemma 7, the answer “YES” at step (2-b-i) is correct. The answer “YES” at step (2-a) or (2-b-iii) is also correct, as  $V(\tau)$  is  $k$ -feasible and hence  $V(G) = W$  is  $k$ -feasible by Lemma 3 in this case. To justify the answer “NO” at step 3, we suppose  $V(G)$  is  $k$ -feasible and show that the algorithm terminates at step (2-a), (2-b-i), or (2-b-iii). Below, we abuse notation and let  $\mathcal{S}_i$ ,  $0 \leq i < n$ , denote the final value of the program variable  $\mathcal{S}_i$ . Since  $V(G)$  is  $k$ -feasible,  $\mathcal{S}_0$  contains a strongly  $k$ -feasible sequence, the empty sequence. Now, suppose  $\sigma \in \mathcal{S}_i$  is strongly  $k$ -feasible for some  $i < n$ . Then, there is some  $v \in V(G) \setminus V(\sigma)$  such that  $\sigma v$  is strongly  $k$ -feasible. If case (2-a) applies to  $\sigma v$ , then  $\tau = \text{push}_k(\sigma v)$ , a strongly  $k$ -feasible sequence strictly longer than  $\sigma$ , is added to  $\mathcal{S}_{|\tau|}$ . Otherwise, (2-b-i) or (2-b-iii) applies to  $\sigma v$ . If case (2-b-i) applies then we are done. So, suppose case (2-b-iii) applies. If  $W = V(G)$  and the algorithm terminates then we are done. Otherwise,  $\mathcal{S}_{i'}$  where  $i' = |W| > i$  contains a strongly  $k$ -feasible sequence. By a straightforward induction, we conclude that the algorithm eventually terminates at step (2-a), (2-b-i), or (2-b-iii).  $\square$

**Lemma 9** *For  $k \leq n/2$ , algorithm SMALL-WIDTH runs in  $O(2^{(H(k/n)+\epsilon)n})$  time.*

*Proof* By Lemma 7, the algorithm spends  $n^{O(g)} = n^{O(1)}$  time to process each element in  $\mathcal{S} = \bigcup_{0 \leq i < n} \mathcal{S}_i$ . Therefore, it is enough to show that  $|\mathcal{S}| = O(2^{(H(k/n)+\epsilon')n})$  for some  $\epsilon' < \epsilon$ .

For each  $0 \leq i \leq n$ , let  $\mathcal{U}_i = \{V(\sigma) \mid \sigma \in \mathcal{S}_i\}$  and  $\mathcal{U} = \bigcup_{1 \leq i < n} \mathcal{U}_i$ . Since the algorithm stores in  $\mathcal{S}_i$  only one  $\sigma$  with  $V(\sigma) = U$  for each  $U$ , we have  $|\mathcal{S}_i| = |\mathcal{U}_i|$  for  $0 \leq i < n$ .

Let  $U \in \mathcal{U}$ . Since  $U = \text{push}_k(X)$  for some  $X$ , we have  $U = \text{push}_k(U)$ . By Lemma 4, for each  $S \subseteq V(G)$  with  $|S| \leq k - g$ , the number of non-empty  $U$  with  $N^-(U) = S$  and  $\text{push}_k(U) = U$  is at most  $2^{\epsilon' n}$  where  $\epsilon' = \frac{1}{g+2} < \epsilon$ . The desired upper bound on  $|\mathcal{S}|$  follows since the number of vertex sets  $S$  with  $|S| \leq k - g$  is smaller than  $2^{H(k/n)n}$ .  $\square$

## 6 Combining the Two Algorithms

Theorem 1 immediately follows from a combination of the two algorithms given in the previous sections. Observe  $2^{H(1/3)} < 1.89$  and choose positive  $\delta$  and  $\epsilon$  so that  $2^{H(1/3+\delta)+\epsilon} < 1.89$ . If  $k > (\frac{1}{3} + \delta)n$ , we apply algorithm LARGE-WIDTH; otherwise, we apply algorithm SMALL-WIDTH. From Lemmas 6 and 9, we see that the running time of the algorithm is  $O(1.89^n)$  in both cases.

## 7 XP Algorithm

In this section, we develop an algorithm based on general commitments to prove Lemma 7. This algorithm also immediately implies an XP algorithm for directed pathwidth.

We need some tools for finding  $k$ -committable extensions and for deciding if a given extension is  $k$ -committable.

**Lemma 10** *Let  $\sigma$  be a  $k$ -feasible sequence for  $G$  and let  $\tau$  be a  $k$ -feasible proper extension of  $\sigma$ . Let  $W$  be a vertex set with  $V(\sigma) \subsetneq W \subseteq V(\tau)$  such that  $d^-(W) \leq d^-(X)$  for every  $X$  with  $V(\sigma) \subseteq X \subseteq V(\tau)$ . Let  $\tau_W$  denote the subsequence of  $\tau$  such that  $V(\tau_W) = W$ . Then,  $\tau_W$  is a  $k$ -committable extension of  $\sigma$ .*

*Proof* From the choice of  $W$ , we have  $d^-(X) \geq d^-(W)$  for every  $X$ ,  $V(\sigma) \subseteq X \subseteq W$ . Therefore, it suffices to show that  $\tau_W$  is  $k$ -feasible. Consider an arbitrary extension  $\alpha$  of  $\sigma$  that is a prefix of  $\tau_W$ . Our goal is to show that  $d^-(\alpha) \leq k$ .

Let  $\beta$  be the shortest prefix of  $\tau$  such that  $V(\alpha) \subseteq V(\beta)$ . By submodularity, we have

$$d^-(\beta) + d^-(W) \geq d^-(\alpha) + d^-(V(\beta) \cup W),$$

where we have used  $V(\alpha) = V(\beta) \cap W$ . Since  $\beta$  is a prefix of  $\tau$ , we have  $d^-(\beta) \leq k$ . Moreover, we have  $d^-(W) \leq d^-(V(\beta) \cup W)$  by the assumption on  $W$  since  $V(\sigma) \subsetneq V(\beta) \cup W \subseteq V(\tau)$ . Therefore, we have  $d^-(\alpha) \leq k$ . Since this holds for every extension  $\alpha$  of  $\sigma$  that is a prefix of  $\tau_W$  and  $\sigma$  is  $k$ -feasible,  $\tau_W$  is  $k$ -feasible.  $\square$

**Corollary 2** *Let  $\sigma$  be a  $k$ -feasible sequence and let  $\tau$  be a  $k$ -feasible proper extension of  $\sigma$  with  $d^-(\tau) \leq d^-(\sigma)$ . Then, there exists a  $k$ -committable extension of  $\sigma$  that is a subsequence of  $\tau$ . Moreover, such a subsequence can be computed in  $O(km)$  time, given  $\sigma$  and  $\tau$ , where  $m$  is the number of edges of  $G$ .*

*Proof* Let  $W$  minimize  $d^-(W)$  subject to  $V(\sigma) \subseteq W \subseteq V(\tau)$ . We may choose  $W$  to be a proper superset of  $V(\sigma)$  since  $d^-(\tau) \leq d^-(\sigma)$ . Lemma 10 applies and  $\tau_W$ , the subsequence of  $\tau$  with  $V(\tau_W) = W$ , is a  $k$ -committable extension of  $\sigma$ .

To obtain such  $W$ , we define digraph  $G'$  as follows:  $V(G') = I \cup \{s, t\}$ , where  $s$  and  $t$  are distinct vertices not in  $V(G)$  and  $I = V(\tau) \cup N^-(\tau) \setminus V(\sigma)$ ;  $E(G') = E(G[I]) \cup \{(s, v) \mid v \in N^-(\tau)\} \cup \{(v, t) \mid v \in N^-(\sigma)\}$ . We say that  $S \subseteq I$  is an  $s$ - $t$  separator if there is no directed path from  $s$  to  $t$  in  $G'[V(G') \setminus S]$ .

Observe that, for every  $U$  such that  $V(\sigma) \subseteq U \subseteq V(\tau)$ ,  $N^-(U)$  is an  $s$ - $t$  separator. Conversely, each minimal  $s$ - $t$  separator  $S$  is  $N^-(U)$  for some  $V(\sigma) \subseteq U \subseteq V(\tau)$ . To see this, let  $S$  be an arbitrary minimal  $s$ - $t$  separator, let  $R_S$  denote the set of vertices  $v \in I$  such that there is a directed path of  $G'[V(G') \setminus S]$  from  $s$  to  $v$ , and let  $U_S = V(\sigma) \cup I \setminus (S \cup R_S)$ . Then, we have  $U_S \subseteq V(\tau)$  since  $N^-(\tau) \subseteq R_S$  and  $V(\tau) = V(\sigma) \cup I \setminus N^-(\tau)$ . Therefore,  $N^-(U_S)$  is an  $s$ - $t$  separator. We must have  $N^-(U_S) \subseteq S$  since a vertex  $v \in N^-(U_S) \setminus S \subseteq R_S$  would provide a directed path from  $s$  to some vertex in  $U_S$ , a contradiction. Since  $S$  is a minimal  $s$ - $t$  separator, we conclude that  $S = N^-(U_S)$ .

Thus, the problem of finding  $W$  such that  $V(\sigma) \subseteq W \subseteq V(\tau)$  minimizing  $|N^-(W)|$  reduces to that of finding a minimum  $s$ – $t$  separator, which can be done by a standard augmenting path algorithm for the maximum number of internally disjoint  $s$ – $t$  paths. If we find  $d^-(\tau)$  internally disjoint paths, then  $N^-(\tau)$  is a minimum  $s$ – $t$  separator and we are done. Otherwise, the algorithm finds a minimum  $s$ – $t$  separator  $S$  with  $|S| < d^-(\tau)$ . We set  $W = U_S$ . Since  $d^-(V(\sigma)) \geq d^-(\tau) > |S|$ , we cannot have  $W = V(\sigma)$  and therefore  $W$  is the required set. Since the disjoint paths computation takes at most  $k$  augmenting steps and each step takes  $O(m)$  time, the time for this computation is  $O(km)$ .  $\square$

We formalize our algorithm as a recursive procedure `Search` below. We consider  $G$  and  $k$  fixed in the algorithm description. Procedure `Search` receives a non-negative integer  $g \leq k$  and a  $k$ -feasible sequence  $\sigma$  such that  $d^-(\sigma) \geq k - g$  and returns one of the following:

1. **(SOLUTION  $\tau$ )**,
2. **FAILURE**, or
3. **(COMMIT  $\tau$ )**.

When call `Search( $g, \sigma$ )` returns **(SOLUTION  $\tau$ )**,  $\tau$  is a  $k$ -feasible permutation that extends  $\sigma$ ; when it returns **FAILURE**, there is no  $k$ -feasible permutation that extends  $\sigma$ ; when it returns **(COMMIT  $\tau$ )**,  $\tau$  is a  $k$ -committable extension of  $\sigma$  satisfying  $d^-(\tau) \leq k - g$ .

We start with an auxiliary procedure `Exhaust`, which does an exhaustive search.

**Procedure** `Exhaust( $\sigma$ )`

**Input:** a  $k$ -feasible sequence  $\sigma$ .

**Output:** **(SOLUTION  $\tau$ )**, where  $\tau$  is a  $k$ -feasible permutation that extends  $\sigma$  or **FAILURE**, when it is found that there is no  $k$ -feasible permutation that extends  $\sigma$ .

**Steps:**

Case E1:  $|\sigma| \geq n - k - 1$ . Arbitrarily extend  $\sigma$  into a permutation  $\sigma'$  and Return **(SOLUTION  $\sigma'$ )**.

Case E2: Otherwise. For each  $v \in V(G) \setminus V(\sigma)$  such that  $d^-(\sigma v) \leq k$ , call `Exhaust( $\sigma v$ )`. If any of these calls returns a result of the form **SOLUTION $\tau$** , return that result. Otherwise, return **FAILURE**.

The correctness of this procedure is straightforward. Note that  $d^-(\sigma') \leq k$  for every proper extension  $\sigma'$  of  $\sigma$  if  $|\sigma| \geq n - k - 1$  and that  $\sigma v$  is a  $k$ -committable extension of  $\sigma$  if  $d^-(\sigma v) \leq d^-(\sigma)$ .

**Proposition 5** *When call `Exhaust( $\sigma$ )` returns **(SOLUTION  $\tau$ )**,  $\tau$  is a  $k$ -feasible permutation. When it returns **FAILURE** then there is no  $k$ -feasible permutation that extends  $\sigma$ .*

For each  $k$ -feasible sequence  $\sigma$ , let  $X(\sigma)$  denote the total number of calls to `Exhaust` generated by the call `Exhaust( $\sigma$ )` (counting this call itself).

**Lemma 11** *Let  $\sigma$  be a  $k$ -feasible sequence and let  $n - |\sigma| = r$ . Suppose  $r \geq k + 1$ . Then, we have*

$$X(\sigma) \leq 2r^{r-k-1} - 1.$$

*Proof* The proof is by induction on  $r$ . The base case  $r = k + 1$  is trivial since  $X(\sigma) = 1$  if  $n - |\sigma| = k + 1$ .

Suppose  $r > k + 1$ . Then, Case E2 applies and  $\text{Exhaust}(\sigma v)$  is called for each  $v \in V(G) \setminus V(\sigma)$  such that  $d^-(\sigma v) \leq k$ . We have  $X(\sigma v) \leq 2(r - 1)^{r-k-2} - 1$  for each  $v \in V(G) \setminus V(\sigma)$  such that  $d^-(\sigma v) \leq k$ , by the induction hypothesis. Therefore we have

$$\begin{aligned} X(\sigma) &\leq 1 + r \left( 2(r - 1)^{r-k-2} - 1 \right) \\ &\leq 2r^{r-k-1} - (r - 1) \\ &\leq 2r^{r-k-1} - 1, \end{aligned}$$

as  $r > k \geq 1$ . □

Our procedure **Search** calls, in addition to **Exhaust**, another auxiliary procedure **Dig**, which in turn calls **Search**.

**Procedure**  $\text{Search}(g, \sigma)$

**Input:** an integer  $g$  and sequence  $\sigma$  that satisfy the following conditions.

IS1:  $0 \leq g \leq k$ ,

IS2:  $\sigma$  is  $k$ -feasible, and

IS3:  $d^-(\sigma) \geq k - g$ .

**Output:** (**SOLUTION**  $\tau$ ), where  $\tau$  is a  $k$ -feasible permutation that extends  $\sigma$ , **FAILURE**, when it is found that there is no  $k$ -feasible permutation that extends  $\sigma$ , or (**COMMIT**  $\tau$ ), where  $\tau$  is a  $k$ -committable extension of  $\sigma$  with  $d^-(\tau) \leq k - g$ .

**Steps:**

Case S1:  $n - |\sigma| \leq k + g + 1$ . Call  $\text{Exhaust}(\sigma)$  and return the result.

Case S2: Otherwise and there is some  $v \in V(G) \setminus V(\sigma)$  such that  $d^-(\sigma v) \leq d^-(\sigma)$ .

Case S2-1:  $d^-(\sigma v) \leq k - g$ . Return (**COMMIT**  $\sigma v$ ).

Case S2-2:  $d^-(\sigma v) > k - g$ . Call  $\text{Dig}(g, \sigma, \sigma v)$  and return the result.

Case S3: Otherwise. For each  $v \in V(G) \setminus V(\sigma)$  such that  $d^-(\sigma v) \leq k$ , call  $\text{Dig}(g, \sigma, \sigma v)$ . Note that we have  $g > 0$  whenever this call occurs, as otherwise  $d^-(\sigma) = k$  and hence Case S2 would apply. If any of these calls returns a result distinct from **FAILURE**, return that result. Otherwise return **FAILURE**.

**Procedure**  $\text{Dig}(g, \sigma, \eta)$

**Input:** an integer  $g$  and sequences  $\sigma$  and  $\eta$  that satisfy the following conditions.

ID1:  $1 \leq g \leq k$ ,

ID2:  $\sigma$  is  $k$ -feasible,

ID3:  $d^-(\sigma) \geq k - g$ ,

ID4:  $\eta$  is a proper  $k$ -feasible extension of  $\sigma$ , and

ID5: for every prefix  $\eta'$  of  $\eta$  with  $|\eta'| > |\sigma|$ , we have  $d^-(\eta') > k - g$ .

**Output:** (**SOLUTION**  $\tau$ ), where  $\tau$  is a  $k$ -feasible permutation that extends  $\eta$ , **FAILURE**, when it is found that there is no  $k$ -feasible permutation that extends  $\eta$ , or (**COMMIT**  $\tau$ ), where  $\tau$  is a  $k$ -committable extension of  $\sigma$  with  $d^-(\tau) \leq k - g$ .

### Steps:

- Case D1:  $n - |\eta| \leq k + g$ . Call  $\text{Exhaust}(\eta)$  and return the result.
- Case D2: Otherwise. Call  $\text{Search}(k - d^-(\eta), \eta)$ .
- Case D2-1: The result of the call is **FAILURE**. Return **FAILURE**.
- Case D2-2: The result of the call is (**SOLUTION**  $\tau$ ). Return (**SOLUTION**  $\tau$ ).
- Case D2-3: The result of the call is (**COMMIT**  $\tau$ ).
- Case D2-3-1:  $d^-(\tau) \leq k - g$ . Apply Corollary 2 to  $\sigma$  and  $\tau$  to find a  $k$ -committable extension  $\tau'$  of  $\sigma$  with  $d^-(\tau') \leq k - g$ . Return (**COMMIT**  $\tau'$ ).
- Case D2-3-2:  $d^-(\tau) > k - g$ . Call  $\text{Dig}(g, \sigma, \tau)$  and return the result.

Here is some intuition behind the algorithm. As described in the output specification, the call  $\text{Search}(g, \sigma)$  tries to either find a  $k$ -feasible permutation that extends  $\sigma$ , conclude that none such exists, or find a  $k$ -committable extension of  $\tau$  with  $d^-(\tau) \leq k - g$ . We need to be careful in dealing with the third type of results and this is indeed the reason for the rather complicated mutual recursion. Suppose a recursive call on  $\sigma v$ ,  $v \in V(G) \setminus V(\sigma)$ , returned (**COMMIT**  $\tau$ ). If  $d^-(\tau) \leq k - g$ , then we may compute a  $k$ -committable extension  $\tau'$  of  $\sigma$  by Corollary 2 and return (**COMMIT**  $\tau'$ ). However, it is possible that  $d^-(\tau) > k - g$  as  $d^-(\sigma v) > d^-(\sigma) \geq k - g$ . In this case, we need to “dig” further on  $\tau$  until we get either the results of the first two types or a  $k$ -committable extension  $\tau''$  with  $d^-(\tau'') \leq k - g$ . We formulate this repetition as an auxiliary recursion by  $\text{Dig}$  in order to facilitate the running time analysis.

We say that an input  $(g, \sigma)$  to  $\text{Search}$  or an input  $(g, \sigma, \eta)$  to  $\text{Dig}$  is *valid* if it satisfies the conditions stated in the **Input** section of the procedure description. We also say that the call  $\text{Search}(g, \sigma)$  or  $\text{Dig}(g, \sigma, \eta)$  is valid in that case. We say that an answer from those calls is *correct* if it satisfies the conditions stated in the **Output** section of the procedure description.

**Lemma 12** *The algorithm is correct: every valid call to  $\text{Search}$  returns a correct answer and every valid call to  $\text{Dig}$  returns a correct answer.*

*Proof* The proof is by induction on  $n - |\sigma| - g$  for  $\text{Search}$  and on  $n - |\eta| - g$  for  $\text{Dig}$ . The base case for  $\text{Search}$  is where  $n - |\sigma| - g \leq k + 1$  and the base case for  $\text{Dig}$  is where  $n - |\eta| - g \leq k$ . In these cases, the task is delegated to procedure  $\text{Exhaust}$  and the result is correct.

For the induction step for  $\text{Search}$ , let  $q > k + 1$  and assume that every valid call  $\text{Dig}(g', \sigma', \eta')$  returns a correct answer if  $n - |\eta'| - g' < q$ . Consider a valid call  $\text{Search}(g, \sigma)$  with  $n - |\sigma| - g = q$ . If there is no  $v \in V(G) \setminus V(\sigma)$  such that  $d^-(\sigma v) \leq k$ , then  $\text{Search}(g, \sigma)$  correctly returns **FAILURE** in Case S3. Suppose that there is some  $v \in V(G) \setminus V(\sigma)$  such that  $d^-(\sigma v) \leq d^-(\sigma)$ . By Lemma 1,  $\sigma v$  is a  $k$ -committable extension of  $\sigma$ . If  $d^-(\sigma v) \leq k - g$ ,  $\text{Search}(g, \sigma)$  correctly returns (**COMMIT**  $\sigma v$ ) in Case S2-1.

Otherwise, call  $\text{Dig}(g, \sigma, \eta)$  is made with  $\eta = \sigma v$  (Case S2-2). We confirm that this call is valid. Condition ID1 holds, since  $g \leq k$  by IS1 and, moreover, we have  $k - g \leq d^-(\sigma) < d^-(\sigma v) \leq k$ , from IS3 and the assumption of Case S2-2, and hence  $g \geq 1$ . Conditions ID2 and ID3 are exactly conditions IS2 and IS3, respectively. Condition ID4 is satisfied because of the assumption  $d^-(\eta) \leq d^-(\sigma)$  of Case S2. Condition ID5 is also satisfied, since  $\eta = \sigma v$  is the only prefix of  $\eta$  that is longer

than  $\sigma$  and  $d^-(\eta) > k - g$  by the assumption of Case S2-2. We moreover have  $n - |\eta| - g = n - |\sigma| - g - 1 < q$ . Therefore, by the induction hypothesis,  $\text{Dig}(g, \sigma, \eta)$  returns a correct answer. Comparing the output specifications of  $\text{Search}$  and  $\text{Dig}$ , we see that this answer is also correct for the call  $\text{Search}(g, \sigma)$ .

Next suppose that  $d^-(\sigma v) > d^-(\sigma)$  for each  $v \in V(G) \setminus V(\sigma)$ . For each such  $v$  with  $d^-(\sigma v) \leq k$ ,  $\text{Dig}(g, \sigma, \sigma v)$  is called (Case S3). The validity of this input is verified similarly as above for Case S2-2. The only difference is that the condition  $d^-(\sigma v) \leq k$  in ID4 is directly assumed in Case 3, rather than inferred from  $d^-(\sigma v) \leq d^-(\sigma)$ . By the induction hypothesis,  $\text{Dig}(g, \sigma, \sigma v)$  returns a correct answer. If this call for any  $v$  returns (**SOLUTION**  $\tau$ ), then  $\text{Search}(g, \sigma)$  correctly returns (**SOLUTION**  $\tau$ ). If this call for any  $v$  returns (**COMMIT**  $\tau$ ), then  $\tau$  is a  $k$ -committable extension of  $\sigma$  with  $d^-(\tau) \leq k - g$  and  $\text{Search}(g, \sigma)$  correctly returns (**COMMIT**  $\tau$ ). If this call returns **FAILURE** for all  $v$ , then  $\text{Search}(g, \sigma)$  correctly returns **FAILURE**. This completes the induction step for  $\text{Search}$ .

For the induction step for  $\text{Dig}$ , let  $q > k$  and assume that every valid call  $\text{Search}(g', \sigma')$  returns a correct answer if  $n - |\sigma'| - g' \leq q$  and every valid call  $\text{Dig}(g', \sigma', \eta')$  returns a correct answer if  $n - |\eta'| - g' < q$ . Consider a call  $\text{Dig}(g, \sigma, \eta)$  with valid input with  $n - |\eta| - g = q$ . Since  $q > k$ , Case D2 applies and  $\text{Search}(g', \eta)$  is called with  $g' = k - d^-(\eta)$ . Since  $\eta$  is  $k$ -feasible by ID4 and hence  $0 \leq d^-(\eta) \leq k$ , we have  $0 \leq g' \leq k$  satisfying IS1 for this call. IS2 is satisfied since  $\eta$  is  $k$ -feasible and IS3 is satisfied since  $d^-(\eta) = k - g'$ . Since  $d^-(\eta) > k - g$  by the input specification of  $\text{Dig}$  and hence  $g' > g$ , we have  $n - |\eta| - g' < q$ . Therefore, we may apply the induction hypothesis to the call  $\text{Search}(g', \eta)$  and have a correct answer. If this answer is **FAILURE** or (**SOLUTION**  $\tau$ ) for some  $\tau$ , then  $\text{Dig}(g, \sigma, \eta)$  correctly returns this answer in Case D2-1 or D2-2. Suppose that the answer is of the form (**COMMIT**  $\tau$ ) (Case D2-3). By the correctness of the call  $\text{Search}(g', \eta)$ ,  $\tau$  is a  $k$ -committable extension of  $\eta$ . If  $d^-(\tau) \leq k - g$ , Case D2-3-1 applies, as  $\eta$  is an extension of  $\sigma$  by ID4, and a  $k$ -committable extension  $\tau'$  of  $\sigma$  with  $d^-(\tau') \leq k - g$  obtained by Corollary 2 is correctly returned.

Suppose finally that  $d^-(\tau) > k - g$ . A further call  $\text{Dig}(g, \sigma, \tau)$  is made in Case D2-3-2. We confirm that this is a valid call. Conditions ID1, ID2 and ID3 for this call are the same as for the call  $\text{Dig}(g, \sigma, \eta)$  and therefore are satisfied. Since  $\eta$  is a proper extension of  $\sigma$  and  $\tau$  is a  $k$ -committable extension of  $\eta$ , Condition ID4 is satisfied. Since  $d^-(\tau) > k - g$ ,  $d^-(\eta) > k - g$ , and  $\tau$  is a  $k$ -committable extension of  $\eta$ , we have  $d^-(\tau') > k - g$  for each prefix  $\tau'$  of  $\tau$  that is an extension of  $\eta$ . Combined with Condition ID5 on  $\eta$  we have  $d^-(\tau') > k - g$  for each prefix  $\tau'$  of  $\tau$  that is a proper extension of  $\sigma$ , Condition ID5 on  $\tau$ . We also have  $n - |\tau| - g < n - |\eta| - g = q$ . Therefore, by the induction hypothesis, the answer of  $\text{Dig}(g, \sigma, \tau)$  is correct. If this answer is (**SOLUTION**  $\alpha$ ) for some  $\alpha$ , then  $\text{Dig}(g, \sigma, \eta)$  correctly returns (**SOLUTION**  $\alpha$ ). If this answer is **FAILURE**, then  $\tau$  does not extend to any  $k$ -feasible permutation. Since  $\tau$  is a  $k$ -committable extension of  $\eta$ , this implies that  $\eta$  does not extend to any  $k$ -feasible permutation either. Therefore, the answer **FAILURE** to the call  $\text{Dig}(g, \sigma, \eta)$  is correct. Finally, if the answer from  $\text{Dig}(g, \sigma, \tau)$  is (**COMMIT**  $\alpha$ ) for some  $\alpha$ ,  $\alpha$  is a  $k$ -committable extension of  $\sigma$  with  $d^-(\alpha) \leq k - g$  by the correctness of this call. Therefore, the answer (**COMMIT**  $\alpha$ ) to the call  $\text{Dig}(g, \sigma, \eta)$  is correct. This completes the induction step for  $\text{Dig}$ .  $\square$

The following straightforward observation based on Cases S1 and D1 of the above algorithm is used in the analysis of the running time below.

**Proposition 6** *When the return value of the call to Search or Dig is (COMMIT  $\tau$ ), we have  $n - |\tau| \geq k + 1$ .*

For each valid input  $(g, \sigma)$  to procedure Search, let  $S(g, \sigma)$  denote the total number of calls to Search and Exhaust generated by the call Search( $g, \sigma$ ) (including this call itself). Similarly, for each valid input  $(g, \sigma, \eta)$  to Dig, let  $D(g, \sigma, \eta)$  denote the total number of calls to Search and Exhaust generated by the call Dig( $g, \sigma, \eta$ ). Note that we do not count the number of calls to Dig, which is no greater than the number of calls to Search and Exhaust.

**Lemma 13** *Let  $P_1(g, r)$  denote the statement that inequality*

$$S(g, \sigma) \leq \frac{2r^{2g}}{g!} \quad (1)$$

*holds for each valid input  $(g, \sigma)$  to procedure Search such that  $|\sigma| = n - r$ .*

*Let  $P_2(g, r)$  denote the statement that inequality*

$$D(g, \sigma, \eta) \leq \frac{2r^{2g-1}}{g!} \quad (2)$$

*holds for each valid input  $(g, \sigma, \eta)$  to procedure Dig such that  $|\eta| = n - r$ .*

*Then,  $P_1(g, r)$  holds for every pair of integers  $r$  and  $g$  such that  $0 \leq g \leq k$  and  $k + 1 \leq r \leq n$ ;  $P_2(g, r)$  holds for every pair of integers  $r$  and  $g$  such that  $1 \leq g \leq k$  and  $k + 1 \leq r \leq n$ .*

*Proof* The proof is by induction on  $r$ . For the base case of  $P_1(g, r)$ , let  $g, r$  be such that  $0 \leq g \leq k$  and  $k + 1 \leq r \leq k + g + 1$ . Suppose  $\sigma$  is a  $k$ -feasible sequence with  $n - |\sigma| = r$ . Then, in processing call Search( $g, \sigma$ ), Case S1 applies and therefore  $S(g, \sigma) \leq X(\sigma) + 1$ . Recall that  $X(\sigma)$  denotes the total number of calls to Exhaust generated by the call Exhaust( $\sigma$ ) (counting this call itself). By Lemma 11, we have

$$S(g, \sigma) \leq 2r^{r-k-1} \leq 2r^g \leq \frac{2r^{2g}}{g!}.$$

For the base case of  $P_2(g, r)$ , let  $g, r$  be such that  $1 \leq g \leq k$  and  $k + 1 \leq r \leq k + g$ . Suppose  $\sigma$  and  $\eta$  are  $k$ -feasible sequences such that  $(g, \sigma, \eta)$  is a valid input to Dig and  $n - |\eta| = r$ . Then, in processing call Dig( $g, \sigma, \eta$ ), Case D1 applies and therefore  $D(g, \sigma, \eta) \leq X(\eta) + 1$ . By Lemma 11, we have

$$D(g, \sigma, \eta) \leq 2r^{r-k-1} \leq 2r^{g-1} \leq \frac{2r^{2g-1}}{g!}.$$

For the induction step on  $P_1(g, r)$ , let  $0 \leq g \leq k$  and  $r > k + g + 1$ . We assume that  $P_1(g', r')$  holds for every pair of  $g'$  and  $r'$  with  $0 \leq g' \leq k$  and  $k + 1 \leq r' < r$  as well as that  $P_2(g', r')$  holds for every pair of  $g'$  and  $r'$  with  $1 \leq g' \leq k$  and  $k + 1 \leq r' < r$ .

Consider a call  $\text{Search}(g, \sigma)$  with a valid input, where  $|\sigma| = n - r$ . Since  $r > k + g + 1$ , Case S2-1, S2-2, or S3 applies. If Case S2-1 applies then  $S(g, \sigma) = 1$  and we are done. Suppose Case S2-2 applies:  $\text{Dig}(g, \sigma, \sigma v)$  is called for some  $v \in V(G) \setminus V(\sigma)$  with  $k - g < d^-(\sigma v) \leq d^-(\sigma)$ . By the induction hypothesis, we have  $D(g, \sigma, \sigma v) \leq \frac{2(r-1)^{2g-1}}{g!}$ . Therefore, we have

$$S(g, \sigma) \leq \frac{2(r-1)^{2g-1}}{g!} + 1 \leq \frac{2r^{2g}}{g!}.$$

Suppose Case S3 applies:  $\text{Dig}(g, \sigma, \sigma v)$  is called for each  $v \in V(G) \setminus V(\sigma)$  such that  $d^-(\sigma v) \leq k$ . Similarly to the above, we have  $D(g, \sigma, \sigma v) \leq \frac{2(r-1)^{2g-1}}{g!}$  for each  $v$  and hence

$$S(g, \sigma) \leq \frac{2r(r-1)^{2g-1}}{g!} + 1 \leq \frac{2r^{2g-1}(r-1) + g!}{g!} \leq \frac{2r^{2g}}{g!}.$$

For the induction step on  $P_2(g, r)$ , let  $g$  and  $r$  be such that  $1 \leq g \leq k$  and  $k + g < r \leq n$ . We assume that  $P_1(g', r')$  holds for every pair of  $g'$  and  $r'$  with  $0 \leq g' \leq k$  and  $k + 1 \leq r' \leq r$  as well as that  $P_2(g', r')$  holds for every pair of  $g'$  and  $r'$  with  $1 \leq g' \leq k$  and  $k + 1 \leq r' < r$ . Consider a call  $\text{Dig}(g, \sigma, \eta)$  on a valid input, where  $|\eta| = n - r$ . Since  $n - |\eta| = r > k + g$ , Case D2 applies.

(Case 1) The return value of the call  $\text{Search}(k - d^-(\eta), \eta)$  is of the form (**SOLUTION**  $\tau$ ) or **FAILURE**. Then the call  $\text{Dig}(g, \sigma, \eta)$  returns immediately. Since  $d^-(\eta) > k - g$ , we have  $k - d^-(\eta) \leq g - 1$  and therefore

$$D(g, \sigma, \eta) \leq S(k - d^-(\eta), \eta) \leq \frac{2r^{2(g-1)}}{(g-1)!} \leq \frac{2r^{2g-1}}{g!}.$$

(Case 2) The return value of the call  $\text{Search}(k - d^-(\eta), \eta)$  is of the form (**COMMIT**  $\tau$ ). Then a recursive call  $\text{Dig}(g, \sigma, \tau)$  is made. Since  $\tau$  is a  $k$ -committable extension of  $\eta$ , we have  $n - |\tau| \leq r - 1$ . Moreover, we have  $n - |\tau| \geq k + 1$  by Proposition 6. Therefore, we may apply the induction hypotheses on  $P_1$  and  $P_2$  and have

$$\begin{aligned} D(g, \sigma, \eta) &\leq S(k - d^-(\eta), \eta) + D(g, \sigma, \tau) \\ &\leq \frac{2r^{2(g-1)}}{(g-1)!} + \frac{2(r-1)^{2g-1}}{g!} \\ &= \frac{2(r-1)^{2g-1} + 2gr^{2g-2}}{g!}. \end{aligned}$$

If  $g = 1$  then we have

$$D(g, \sigma, \eta) \leq 2(r-1) + 2 = 2r = \frac{2r^{2g-1}}{g!}.$$

If  $g > 1$  then, taking the first three terms in the binomial expansion of  $(r - 1)^{2g-1}$ , we have

$$\begin{aligned} D(g, \sigma, \eta) &\leq \frac{2(r^{2g-1} - (2g-1)r^{2g-2} + (2g-1)(g-1)r^{2g-3} + gr^{2g-2})}{g!} \\ &= \frac{2(r^{2g-1} - (g-1)(r^{2g-2} - (2g-1)r^{2g-3}))}{g!} \\ &\leq \frac{2r^{2g-1}}{g!}, \end{aligned}$$

as  $r \geq k + g \geq 2g$ .

This completes the proof of the induction step and hence the proof of the statements of the lemma.  $\square$

Lemma 7 is a straightforward consequence of Lemma 13. The algorithm claimed in Lemma 7 simply makes a call  $\text{Search}(g, \sigma)$ . If the result is of the form (**SOLUTION**  $\tau$ ) then the algorithm concludes that  $\sigma$  is strongly  $k$ -feasible (the first case). If the result is **FAILURE**, then the algorithm concludes that  $\sigma$  is not strongly  $k$ -feasible (the second case). Otherwise, the result must be of the form (**COMMIT**  $\tau$ ), where  $\tau$  is a  $k$ -committable extension of  $\sigma$  with  $d^-(\tau) \leq k - g$  (the third case).

The running time of this algorithm is at most

$$\frac{2(n - |\sigma|)^{2g}}{g!} n^{O(1)} = n^{2g+O(1)},$$

as claimed in Lemma 7.

Another straightforward consequence of Lemma 13 is an XP algorithm for the directed pathwidth.

**Theorem 2** *There is an algorithm that, given a positive integer  $k$  and a digraph  $G$  with  $n$  vertices and  $m$  edges, decides if the directed pathwidth of  $G$  is at most  $k$  in  $O(\frac{mn^{2k}}{(k-1)!})$  time.*

*Proof* We may assume that  $G$  is strongly connected, as otherwise we may apply the algorithm separately to each strongly connected component. The algorithm calls  $\text{Search}(k, \lambda)$ , where  $\lambda$  is the empty sequence. From the assumption that  $G$  is strongly connected, the only  $k$ -committable extensions of  $\lambda$  are permutations. Since each  $k$ -committable extension that may be returned by a call to  $\text{Search}$  has length smaller than  $n - k$  by Proposition 6, it follows the call  $\text{Search}(k, \lambda)$  returns either **FAILURE** or a  $k$ -feasible permutation, deciding whether  $\text{dvsn}(G) \leq k$  or not. By Lemma 13 the number of recursive calls generated by this call is  $O(\frac{2n^{2k}}{k!})$ . The time directly associated to each of such recursive calls is dominated by the time for finding a  $k$ -committable extension using Corollary 2 and is  $O(km)$ . Therefore, the total running time is  $O(\frac{mn^{2k}}{(k-1)!})$ .  $\square$

## References

1. Barát, J.: Directed path-width and monotonicity in digraph searching. *Gr. Comb.* **22**(2), 161–172 (2006)
2. Bodlaender, H.L.: A tourist guide through treewidth. *Acta Cybern.* **11**(1–2), 1–23 (1993)
3. Bodlaender, H.L.: A linear-time algorithm for finding tree-decompositions of small treewidth. *SIAM J. Comput.* **25**(6), 1305–1317 (1996)
4. Bodlaender, H.L., Fomin, F.V., Koster, A.M.C.A., Kratsch, D., Thilikos, D.: A note on exact algorithms for vertex ordering problems on graphs. *Theory Comput. Syst.* **50**(3), 420–432 (2012)
5. Bodlaender, H.L., Kloks, T., Kratsch, D.: Treewidth and pathwidth of permutation graphs. *SIAM J. Discrete Math.* **8**(4), 606–616 (1995)
6. Bodlaender, H.L., Möhring, R.H.: The pathwidth and treewidth of cographs. *SIAM J. Discrete Math.* **6**(2), 181–188 (1992)
7. Coudert, D., Mazauric, D., Nisse, N.: Experimental evaluation of a branch and bound algorithm for computing pathwidth. In: *Proceedings of the 13th International Symposium on Experimental Algorithms (SEA2014)*, Lecture Notes in Computer Science, vol. 8504, pp. 46–58 (2014)
8. Fomin, F.V., Giannopoulou, A.C., Pilipczuk, P.: Computing tree-depth faster than  $2^n$ . In: *Proceedings of the 8th International Symposium on Parameterized and Exact Computation (IPEC2013)*, Lecture Notes in Computer Science, vol. 8246, pp. 137–149 (2013)
9. Fomin, F.V., Kratsch, D.: *Exact Exponential Algorithms*. Springer, Berlin (2010)
10. Fomin, F.V., Villanger, Y.: Finding induced subgraphs via minimal triangulations. In: *Proceedings of the 27th International Symposium on Theoretical Aspects of Computer Sciences (STACS2010)*, Leibniz International Proceedings in Informatics, vol. 5, pp. 383–394 (2010)
11. Gusted, J.: On the pathwidth of chordal graphs. *Discrete Appl. Math.* **45**(3), 233–248 (1993)
12. Habib, M., Möhring, R.H.: Treewidth of cocomparability graphs and a new order-theoretic parameter. *Order* **11**(1), 44–60 (1994)
13. Johnson, T., Robertson, N., Seymour, P.D., Thomas, R.: Directed tree-width. *J. Comb. Theory Ser. B* **82**(1), 138–154 (2001)
14. Kashiwabara, T., Fujisawa, T.: NP-completeness of the problem of finding a minimum-clique-number interval graph containing a given graph as a subgraph. In: *Proceedings of International Symposium on Circuits and Systems*, pp. 657–660 (1979)
15. Kinnarsley, G.N.: The vertex separation number of a graph equals its path-width. *Inf. Process. Lett.* **42**(6), 345–350 (1992)
16. Kitsunai, K., Kobayashi, Y., Komuro, K., Tamaki, H., Tano, T.: Computing directed pathwidth in  $O(1.89^n)$  time. In: *Proceedings of the 7th International Symposium on Parameterized and Exact Computation (IPEC2012)*, Lecture Notes in Computer Science, vol. 7535, pp. 182–193 (2012)
17. Kobayashi, Y., Komuro, K., Tamaki, H.: Search space reduction through commitments in pathwidth computation: An experimental study. In: *Proceedings of the 13th International Symposium on Experimental Algorithms (SEA2014)*, Lecture Notes in Computer Science, vol. 8504, pp. 388–399 (2014)
18. Kloks, T., Bodlaender, H.L., Müller, H., Kratsch, D.: Computing treewidth and minimum fill-in: All you need are the minimal separators. In: *Proceedings of the 1st Annual European Symposium on Algorithms (ESA1993)*, Lecture Notes in Computer Science, vol. 726, pp. 260–271 (1993)
19. Monien, B., Sudborough, I.H.: Min cut is NP-complete for edge weighted trees. *Theor. Comput. Sci.* **58**(1–3), 209–229 (1988)
20. Nagamochi, H.: Linear layouts in submodular systems. In: *Proceedings of the 23rd International Symposium on Algorithms and Computation (ISAAC2012)*, Lecture Notes in Computer Science, vol. 7676 pp. 475–484 (2012)
21. Robertson, N., Seymour, P.D.: Graph minors. I. Excluding a forest. *J. Comb. Theory Ser. B* **35**(1), 39–61 (1983)
22. Robertson, N., Seymour, P.D.: Graph minors XIII. The disjoint paths problem. *J. Comb. Theory Ser. B* **63**(1), 65–110 (1995)
23. Suchan, K., Todinca, I.: Pathwidth of circular-arc graphs. In: *Proceedings of 33rd International Workshop on Graph-Theoretic Concepts in Computer Science (WG2007)*, Lecture Notes in Computer Science, vol. 4769, pp. 258–269 (2007)
24. Suchan, K., Villanger, Y.: Computing pathwidth faster than  $2^n$ . In: *Proceedings of the 4th International Workshop on Parameterized and Exact Computation (IWPEC2009)*, Lecture Notes in Computer Science, vol. 5917, pp. 324–335 (2009)

25. Tamaki, H.: A polynomial time algorithm for bounded directed pathwidth. In: Proceedings of the 37th International Workshop on Graph-Theoretic Concepts in Computer Science (WG2011), Lecture Notes in Computer Science, vol. 6986, pp. 331–342 (2011)
26. Yang, B., Cao, Y.: Digraph searching, directed vertex separation and directed pathwidth. *Discrete Appl. Math.* **156**(10), 1822–1837 (2008)